

Universidad del Pacífico



Portafolio de Criptomonedas

Autores:

Angie Melissa Uchuypoma Romero - am.uchuypomar@alum.up.edu.pe

Karin Mateo Gadea Liza - km.gadeal@alum.up.edu.pe

Rosa Leticia Ortiz Palomino - rl.ortizp@alum.up.edu.pe

Marianela Alison Usnayo Gutierrez - ma.usnayog@alum.up.edu.pe

Docente:

Oswaldo Jose Velasquez Catañon

Curso:

Programación Avanzada para la Ciencia de Datos

Lima, 2026

Índice

Índice.....	2
Introducción.....	2
Marco teórico.....	3
Mercado de criptomonedas.....	3
Retornos y métricas de riesgo.....	3
Teoría de portafolios de Markowitz.....	3
Integración de datos en tiempo real.....	4
Datos y preprocesamiento.....	4
Descripción del dataset histórico.....	4
Pipeline de limpieza de datos.....	4
Construcción de la matriz de retornos.....	4
Datos en tiempo real (CoinGecko API).....	4
Implementación técnica.....	4
Arquitectura del proyecto.....	4
Optimización del portafolio.....	6
Módulo de visualización.....	7
Gestión de estado y caché.....	8
Testing.....	8
Resultados y análisis.....	9
Análisis exploratorio de precios.....	9
Análisis de correlaciones.....	10
Portafolios óptimos por perfil de riesgo.....	11
Backtesting del portafolio óptimo.....	12
Precios en tiempo real.....	13
Conclusión.....	14
Trabajo futuro.....	15
Referencias bibliográficas.....	16
Anexo.....	16

Introducción

En la última década, el mercado de criptomonedas ha experimentado un crecimiento sin precedentes, pasando de ser un fenómeno marginal a convertirse en una clase de activo reconocida globalmente. Bitcoin, la primera criptomoneda descentralizada, fue creada en 2009 y durante años operó en nichos tecnológicos con precios cercanos a cero. Sin embargo, a partir de 2013 comenzó una expansión sostenida del ecosistema crypto que culminó en el rally histórico de 2017, cuando Bitcoin alcanzó los \$20,000 USD y el mercado total de criptomonedas superó los \$800,000 millones de dólares en capitalización. Este auge atrajo a millones de inversionistas minoristas de todo el mundo, muchos de ellos sin experiencia previa en mercados financieros. El problema central que motiva este proyecto es

la asimetría de herramientas entre los grandes actores del mercado y el inversionista individual. Los fondos institucionales y empresas de trading algorítmico cuentan con equipos de analistas cuantitativos, acceso a datos de alta frecuencia y software especializado para construir y optimizar portafolios. El inversionista minorista, en cambio, generalmente toma decisiones basadas en noticias, tendencias en redes sociales o intuición, sin un marco cuantitativo que le permita evaluar el riesgo real de su exposición al mercado. Esta situación se agrava por las características intrínsecas del mercado cripto: opera las 24 horas del día los 7 días de la semana, sin cierres ni interrupciones de circuito como los que existen en bolsas tradicionales. La volatilidad anualizada de Bitcoin, por ejemplo, supera frecuentemente el 80%, frente al 15–20% típico del índice S&P 500. Monero y Dash, otras criptomonedas incluidas en este proyecto, presentaron volatilidades aún mayores durante el período estudiado. Esta volatilidad extrema hace que la diversificación no sea un lujo sino una necesidad: concentrar el capital en un solo activo cripto expone al inversionista a pérdidas catastróficas, como demostró el crash de 2018, cuando el mercado perdió más del 80% de su valor máximo en menos de doce meses. Frente a este escenario, existe una necesidad concreta de herramientas accesibles que apliquen métodos cuantitativos rigurosos al análisis de portafolios de criptomonedas. El presente proyecto responde a esa necesidad mediante el desarrollo de un dashboard interactivo que combina análisis histórico de datos, optimización matemática de portafolios y consulta de precios en tiempo real, poniendo al alcance del usuario técnicas que hasta ahora estaban reservadas para entornos profesionales.

Objetivos del proyecto

Desarrollar un dashboard interactivo para el análisis y la optimización de portafolios de criptomonedas, que integre datos históricos de mercado con precios en tiempo real y aplique el modelo de Media-Varianza de Markowitz para recomendar asignaciones de capital según el perfil de riesgo del usuario.

Objetivos específicos

1. Implementar un pipeline de carga, limpieza y transformación de datos sobre el dataset histórico de CoinMarketCap (Kaggle, CC0), que abarque el período abril 2013 – noviembre 2018 para las cinco criptomonedas de mayor relevancia histórica: Bitcoin, Ethereum, Litecoin, Dash y Monero.
2. Calcular métricas descriptivas de riesgo y retorno por activo, incluyendo retornos logarítmicos diarios, volatilidad anualizada, Sharpe ratio y matriz de correlación entre monedas, como base del análisis exploratorio.
3. Aplicar el modelo de optimización de portafolio de Markowitz mediante el método numérico SLSQP de la librería SciPy, generando portafolios óptimos para tres perfiles de riesgo

diferenciados (conservador, balanceado y agresivo) y trazando la frontera eficiente correspondiente.

4. Integrar datos de precios en tiempo real a través de la API pública de CoinGecko, permitiendo al usuario comparar el comportamiento histórico del mercado con las condiciones actuales y simular el rendimiento de un portafolio en los últimos 30 o 90 días.
5. Construir una interfaz de usuario interactiva, modular y reproducible con Streamlit y Plotly, organizada en pestañas temáticas (Precios & Mercado, Portafolio Óptimo, Live CoinGecko y Datos), que permita al usuario explorar los resultados de forma visual e intuitiva sin necesidad de conocimientos de programación.

Marco teórico

Mercado de criptomonedas

a) Definición y fundamentos

Una criptomoneda es un activo digital descentralizado que utiliza criptografía de clave pública para garantizar la seguridad de las transacciones y controlar la emisión de nuevas unidades. A diferencia del dinero fiduciario emitido por bancos centrales, las criptomonedas operan sobre redes distribuidas basadas en tecnología blockchain: una cadena de bloques inmutable y verificada colectivamente por los participantes de la red, sin necesidad de una autoridad central que valide las transacciones.

Bitcoin, creado en 2009 por el seudónimo Satoshi Nakamoto, fue la primera implementación exitosa de este concepto. Su protocolo establece un suministro máximo de 21 millones de unidades, lo que le confiere una propiedad deflacionaria por diseño. A partir de Bitcoin surgieron miles de proyectos alternativos, denominados altcoins, cada uno con características técnicas, casos de uso y comunidades distintas.

b) Las cinco criptomonedas del proyecto

El dataset utilizado en este proyecto contiene datos históricos de cinco criptomonedas seleccionadas por su relevancia histórica y la disponibilidad continua de datos desde etapas tempranas del mercado: Bitcoin (BTC) es la criptomoneda de mayor capitalización de mercado y la referencia del ecosistema. Funciona como reserva de valor y medio de intercambio, y su comportamiento de precio tiende a influir sobre el resto del mercado. Durante el período estudiado pasó de cotizarse por debajo de \$100 USD en 2013 a alcanzar casi \$20,000 USD en diciembre de 2017.

Ethereum (ETH) fue lanzado en 2015 e introdujo los contratos inteligentes: programas autoejecutables que corren sobre la blockchain sin intermediarios. Esta funcionalidad lo convirtió en la plataforma

base de la mayoría de las aplicaciones descentralizadas (DApps) y tokens ERC-20, diferenciándolo técnicamente de Bitcoin.

Litecoin (LTC) fue creado en 2011 como una versión más rápida y ligera de Bitcoin, con tiempos de confirmación de bloque cuatro veces menores y un suministro máximo de 84 millones de unidades. Históricamente ha mantenido una correlación alta con Bitcoin, aunque con mayor volatilidad relativa. Dash (DASH) incorpora características de privacidad opcional mediante su función PrivateSend y un mecanismo de gobernanza descentralizada a través de masternodes. Fue una de las altcoins de mayor capitalización entre 2016 y 2018.

Monero (XMR) está diseñado específicamente para la privacidad total de las transacciones, utilizando firmas de anillo y direcciones ocultas (stealth addresses) para hacer las transacciones imposibles de rastrear en la blockchain. Esta característica diferencial le otorgó una base de usuarios y un comportamiento de precio relativamente independiente del resto del mercado.

c) Características del mercado crypto respecto a mercados tradicionales

El mercado de criptomonedas presenta diferencias estructurales fundamentales respecto a los mercados financieros tradicionales que tienen implicaciones directas sobre el análisis cuantitativo:

Operación continua. El mercado crypto opera las 24 horas del día, los 7 días de la semana, los 365 días del año, sin cierres, feriados ni interrupciones de circuito. Esto genera series de tiempo sin brechas de fin de semana ni gaps de apertura, lo que simplifica el análisis de retornos diarios pero implica que los períodos de estrés del mercado se propagan sin pausa.

Volatilidad extrema. La volatilidad anualizada de Bitcoin durante el período 2013–2018 superó el 80% en varios subperíodos, llegando a valores por encima del 100% durante los crashes de 2013–2014 y 2018. Esto contrasta drásticamente con el S&P 500, cuya volatilidad histórica anualizada se ubica entre el 15% y el 20%. Las altcoins estudiadas presentaron volatilidades aún mayores, lo que hace que la gestión del riesgo mediante diversificación sea especialmente relevante en este mercado.

Alta correlación en períodos de estrés. Aunque en condiciones normales las criptomonedas pueden presentar correlaciones moderadas entre sí, durante los crashes del mercado la correlación tiende a aumentar bruscamente hacia 1.0, reduciendo los beneficios de diversificación precisamente cuando más se los necesita. Este fenómeno, conocido como correlación condicional, es una limitación conocida del modelo de Markowitz aplicado a activos crypto.

Ausencia de fundamentos convencionales. A diferencia de las acciones, que pueden valorarse mediante flujos de caja descontados, o los bonos, cuyo precio depende de tasas de interés y calidad

crediticia, las criptomonedas carecen de métricas de valoración fundamentales universalmente aceptadas. Sus precios están determinados principalmente por la oferta y demanda, el sentimiento del mercado, los ciclos de adopción tecnológica y los eventos regulatorios.

Fragmentación del mercado. Las criptomonedas se negocian simultáneamente en cientos de exchanges en todo el mundo, con diferencias de precio (arbitraje) entre plataformas. Los datos históricos de CoinMarketCap utilizados en este proyecto representan promedios ponderados entre exchanges, lo que proporciona una referencia de precio de mercado razonablemente representativa.

Retornos y métricas de riesgo

a) Retorno simple y retorno logarítmico

El retorno de un activo financiero mide el cambio porcentual en su precio entre dos períodos consecutivos. Existen dos formas principales de calcularlo, cada una con propiedades distintas que determinan su idoneidad según el contexto de análisis.

El *retorno simple* en el período t se define como:

$$R_T = \frac{P_t - P_{t-1}}{P_{t-1}}$$

donde P_t es el precio de cierre en el día t y P_{t-1} es el precio de cierre en el día anterior. El retorno simple tiene la ventaja de ser intuitivo (un retorno del 10% significa que el precio subió un 10%) y de ser aditivo entre activos para un mismo período, lo que facilita el cálculo del retorno de un portafolio como la suma ponderada de los retornos individuales.

El *retorno logarítmico* (o log-retorno) se define como:

$$r_T = \ln\left(\frac{P_t}{P_{t-1}}\right) = \ln(P_t) - \ln(P_{t-1})$$

El log-retorno presenta tres propiedades que lo hacen preferible para el análisis cuantitativo de series de tiempo financieras:

En primer lugar, es *aditivo en el tiempo*: el retorno logarítmico acumulado entre los períodos 0 y T es simplemente la suma de los log-retornos diarios, es decir, $r_{\{0,T\}} = \sum r_t$. Esta propiedad no se cumple para los retornos simples, cuya composición requiere multiplicación.

En segundo lugar, presenta una distribución *más cercana a la normal*. Dado que los precios no pueden ser negativos, los retornos simples están acotados inferiormente por -1 (una pérdida total del capital), lo que genera distribuciones asimétricas. Los log-retornos, al transformar los precios mediante logaritmo, producen distribuciones más simétricas y manejables estadísticamente.

En tercer lugar, es *numéricamente más estable* para la optimización, especialmente cuando los precios varían en varios órdenes de magnitud, como ocurre con criptomonedas que pasaron de cotizar en centavos a miles de dólares.

Por estas razones, el modelo implementado en este proyecto utiliza retornos logarítmicos diarios como base para todas las métricas y el proceso de optimización.

b) Volatilidad

La volatilidad de un activo financiero es la medida estándar de su riesgo de mercado. Se define como la desviación estándar de sus retornos durante un período determinado. Para datos diarios, la volatilidad diaria σ_d se estima como:

$$\sigma_d = \sqrt{\frac{1}{n-1}} \left(\sum_{t=1}^n (r_t - \bar{r})^2 \right)^{1/2}$$

donde r_t son los log-retornos diarios y $\bar{r}$ es su media.

Para comparar activos o portafolios con diferentes frecuencias de datos, es estándar anualizarla. Dado que el mercado crypto opera 365 días al año (a diferencia de los mercados de acciones que usan 252 días hábiles), la volatilidad anualizada se obtiene multiplicando por la raíz cuadrada de 365:

$$\sigma_{anual} = \sigma_d(\sqrt{365})$$

Esta fórmula asume que los retornos diarios son independientes e idénticamente distribuidos (i.i.d.), supuesto que es una aproximación razonable para períodos cortos pero que puede fallar en presencia de autocorrelación o agrupamiento de volatilidad (volatility clustering), fenómeno común en mercados financieros.

c) Sharpe Ratio

El Sharpe ratio es la métrica más utilizada para evaluar el rendimiento ajustado por riesgo de un portafolio. Fue desarrollado por William F. Sharpe en 1966 y mide cuántas unidades de retorno en exceso sobre la tasa libre de riesgo se obtienen por cada unidad de riesgo asumida:

$$S = \frac{R_p - R_f}{\sigma_p}$$

donde R_p es el retorno anualizado del portafolio, R_f es la tasa libre de riesgo anualizada (en el dashboard, configurable por el usuario; por defecto 0%) y σ_p es la volatilidad anualizada del portafolio.

Un Sharpe ratio mayor indica una mejor compensación riesgo-retorno. Un valor por encima de 1.0 se considera aceptable, por encima de 2.0 se considera muy bueno, y por encima de 3.0 es excepcional. En mercados cripto, los valores pueden ser extremadamente altos durante períodos alcistas y profundamente negativos durante crashes, lo que refleja la naturaleza asimétrica de estos activos.

La tasa libre de riesgo (R_f) representa el retorno que puede obtenerse sin asumir riesgo de mercado, típicamente aproximado por bonos del Tesoro de EE.UU. a corto plazo (T-Bills). Para el período histórico estudiado, la tasa de los T-Bills de 3 meses se ubicó en torno al 1–2% anual.

d) Matriz de covarianza y correlación

La covarianza entre dos activos i y j mide la tendencia de sus retornos a moverse conjuntamente:

$$\sigma_{ij} = \frac{1}{n-1} \left(\sum_{t=1}^n (r_{i,t} - \bar{r}_i)(r_{j,t} - \bar{r}_j) \right)$$

La *matriz de covarianza* Σ agrupa todas las covarianzas entre pares de activos del portafolio, con las varianzas individuales en la diagonal principal. Es la pieza central del modelo de Markowitz, ya que captura no solo el riesgo individual de cada activo sino también cómo se relacionan entre sí.

La *correlación* entre dos activos es la versión normalizada de la covarianza:

$$\rho_{i,j} = \frac{\sigma_{i,j}}{\sigma_i \sigma_j}$$

La correlación toma valores entre -1 y 1. Un valor de 1 indica que los activos se mueven perfectamente al unísono, un valor de -1 indica movimiento perfectamente opuesto, y un valor de 0 indica independencia estadística. Para los fines de la diversificación de portafolios, son especialmente valiosos los pares de activos con correlación baja o negativa, ya que su combinación reduce la varianza total del portafolio.

En el dashboard, la matriz de correlación se visualiza como un heatmap de Plotly donde colores más cálidos indican mayor correlación positiva y colores más fríos indican baja o negativa correlación, permitiendo identificar visualmente las oportunidades de diversificación.

Teoría de portafolios de Markowitz

a) Fundamentos del modelo Media-Varianza

La Teoría Moderna de Portafolios (Modern Portfolio Theory, MPT) fue desarrollada por Harry Markowitz en su artículo seminal "Portfolio Selection" publicado en el Journal of Finance en 1952. Su contribución principal fue demostrar matemáticamente que el riesgo de un portafolio no es simplemente la suma ponderada de los riesgos individuales de sus activos, sino que depende de las correlaciones entre ellos. Esta observación fundamenta el concepto de diversificación como herramienta de gestión de riesgo.

El modelo parte de dos supuestos centrales sobre los inversionistas: son racionales (prefieren mayor retorno para el mismo riesgo) y son aversos al riesgo (ante dos portafolios con el mismo retorno esperado, prefieren el de menor varianza). Bajo estos supuestos, el problema de optimización de portafolios puede formularse de manera precisa.

Formulación matemática del problema

Dado un universo de n activos, se define el vector de pesos $w = (w_1, w_2, \dots, w_n)$ donde w_i representa la fracción del capital invertida en el activo i . El retorno esperado del portafolio es:

$$R_p = W^T u = \sum_{i=1}^n w_i u_i$$

donde μ_i es el retorno esperado anualizado del activo i , estimado a partir del promedio histórico de sus log-retornos multiplicado por 365.

La varianza del portafolio, que representa su riesgo, es:

$$\sigma_p^2 = w^T \Sigma w = \sum_{i=1}^n \sum_{j=1}^n w_i w_j \sigma_{ij}$$

donde Σ es la matriz de covarianza anualizada de los retornos.

El problema de optimización en su versión de mínima varianza se plantea como: $\min_w \sqrt{w^T \Sigma w}$

b) La frontera eficiente

El conjunto de todos los portafolios que minimizan la varianza para cada nivel de retorno esperado forma la frontera eficiente (efficient frontier). En un gráfico de volatilidad (eje X) versus retorno

esperado (eje Y), la frontera eficiente aparece como una curva cóncava hacia la izquierda: cada punto sobre ella representa el portafolio de menor riesgo posible para ese nivel de retorno.

Los portafolios por debajo de la frontera eficiente son ineficientes: existe otro portafolio con el mismo riesgo pero mayor retorno, o con el mismo retorno pero menor riesgo. Un inversionista racional nunca debería mantener un portafolio ineficiente.

En el dashboard, la frontera eficiente se construye resolviendo el problema de optimización 60 veces, barriendo retornos objetivo entre el mínimo y el máximo posible (determinados por los retornos anuales esperados de las monedas individuales). Para cada retorno objetivo se añade una restricción adicional: $w'\mu = R_{\text{objetivo}}$. Los 60 pares (volatilidad, retorno) resultantes se grafican en el plano riesgo-retorno mediante Plotly.

c) **Método de optimización numérica: SLSQP**

Dado que la función objetivo es no lineal (involucra una raíz cuadrada de una forma cuadrática), el problema se resuelve mediante métodos numéricos. El modelo utiliza el algoritmo SLSQP (Sequential Least Squares Programming), implementado en la función `scipy.optimize.minimize` de Python.

SLSQP es un método de programación cuadrática secuencial que resuelve iterativamente subproblemas cuadráticos aproximados del problema original, respetando las restricciones de igualdad (suma de pesos = 1) y desigualdad (pesos ≥ 0) en cada iteración. Se configura con una tolerancia de convergencia `ftol=1e-12` y un máximo de 1,000 iteraciones, partiendo de un punto inicial de pesos iguales ($w_i = 1/n$ para todo i).

d) **Limitaciones conocidas del modelo de Markowitz**

Si bien el modelo de Markowitz es el punto de partida estándar en la teoría de portafolios, presenta limitaciones que son especialmente relevantes en el contexto de criptomonedas:

Sensibilidad a los estimadores. Los pesos óptimos son muy sensibles a pequeños cambios en los retornos esperados y la matriz de covarianza. En la práctica, estimar los retornos futuros a partir de promedios históricos es notoriamente impreciso, lo que puede generar portafolios óptimos "en papel" con pobre desempeño real fuera de la muestra.

Supuesto de normalidad. El modelo asume implícitamente que los retornos siguen una distribución normal. Los retornos de criptomonedas exhiben leptocurtosis (colas más pesadas que la normal) y asimetría negativa, lo que significa que los eventos extremos (crashes) son más frecuentes de lo que el modelo predice.

Correlaciones no estacionarias. El modelo asume que la matriz de covarianza es estable a lo largo del tiempo. Sin embargo, las correlaciones entre criptomonedas varían significativamente según el régimen de mercado (alcista o bajista), limitando la validez de estimaciones basadas en promedios históricos.

Integración de datos en tiempo real

a) APIs REST y su rol en el análisis financiero

Una API (Application Programming Interface) REST (Representational State Transfer) es una interfaz de comunicación entre sistemas que permite a una aplicación cliente solicitar datos a un servidor remoto mediante el protocolo HTTP. Las APIs REST son el estándar predominante para el intercambio de datos en tiempo real en aplicaciones financieras, ya que permiten obtener precios actualizados, métricas de mercado e información histórica reciente sin necesidad de descargar datasets completos.

En el contexto de este proyecto, la integración de una API de datos cripto en tiempo real cumple una función doble: enriquece el análisis al complementar los datos históricos del dataset de Kaggle (que llega hasta noviembre de 2018) con información del mercado actual, y demuestra una capacidad técnica esencial en ciencia de datos aplicada al análisis financiero: la construcción de pipelines que consumen datos de fuentes externas en producción.

b) CoinGecko API

CoinGecko es una de las principales plataformas de seguimiento de datos del mercado de criptomonedas, con cobertura de más de 10,000 activos en más de 700 exchanges. Su API pública es especialmente adecuada para este proyecto por tres razones:

Acceso gratuito sin autenticación. El nivel público de la API no requiere registro ni clave de API, lo que elimina barreras de acceso y permite que el dashboard funcione de forma inmediata en cualquier entorno sin configuración adicional.

Endpoints relevantes para el proyecto. El endpoint `/simple/price` permite obtener en una sola solicitud el precio actual en USD, la variación porcentual en las últimas 24 horas, el volumen de transacciones y la capitalización de mercado para múltiples monedas simultáneamente. El endpoint `/coins/{id}/market_chart` permite descargar series de precios históricos recientes con granularidad diaria para los últimos 30 o 90 días.

Estabilidad y documentación. CoinGecko mantiene una API bien documentada y con alta disponibilidad, adecuada para un proyecto académico que no requiere datos de alta frecuencia ni latencia mínima garantizada.

c) Manejo del rate limiting

La versión pública de la API de CoinGecko aplica un límite de solicitudes por minuto (rate limit) para evitar el abuso del servicio. Para cumplir con este límite, el módulo `processing.py` implementa una pausa de 1.1 segundos entre solicitudes consecutivas mediante `time.sleep(1.1)` cuando se realizan múltiples llamadas al endpoint de series históricas (una por cada moneda seleccionada).

Adicionalmente, el dashboard utiliza el decorador `@st.cache_data` de Streamlit para almacenar en caché los resultados de las consultas a CoinGecko durante la sesión de usuario. Esto significa que la API solo se consulta una vez por sesión (o cuando el usuario presiona explícitamente el botón "Actualizar precios"), evitando solicitudes redundantes y mejorando la experiencia de uso.

d) Arquitectura del flujo de datos

El proyecto integra dos fuentes de datos con características y propósitos distintos que se complementan:

Los datos históricos provienen del archivo `crypto-markets.csv` (CoinMarketCap via Kaggle, licencia CC0). Este dataset es estático, local, no requiere conexión a internet y cubre un período amplio (2013–2018) que permite estimar retornos y covarianzas con una muestra estadísticamente significativa. Es la base del análisis de portafolio y del backtesting.

Los datos en tiempo real provienen de la API de CoinGecko y se obtienen bajo demanda. Cubren las condiciones actuales del mercado y los últimos 30 o 90 días, permitiendo al usuario comparar las condiciones históricas con el contexto presente. Estos datos se utilizan en la pestaña "Live CoinGecko" del dashboard para el gráfico de precios recientes y los KPIs actuales (precio, variación 24h, market cap, volumen).

Esta arquitectura dual garantiza que el dashboard sea funcional incluso sin conexión a internet (utilizando únicamente los datos históricos locales) y que incorpore información actualizada cuando la conexión está disponible, combinando robustez y relevancia en el análisis.

Datos y preprocesamiento

Descripción del dataset histórico

El dataset histórico utilizado proviene de **CoinMarketCap Historical Data**, publicado en Kaggle bajo licencia CC0 (dominio público).

El archivo crudo (`crypto-markets.csv`) contiene 942,297 registros correspondientes a 2,071 criptomonedas distintas, con 13 columnas: `slug`, `symbol`, `name`, `date`, `ranknow`, `open`, `high`, `low`, `close`, `volume`, `market`, `close_ratio` y `spread`. Para este proyecto se trabajó únicamente con las ocho variables relevantes para el análisis financiero (`name`, `date`, `open`, `high`, `low`, `close`, `volume`, `market`), validadas

en la función `cargar_datos()`, que verifica su presencia antes de continuar y lanza un error si falta alguna.

Del universo total de monedas se seleccionaron cinco activos por su relevancia histórica y disponibilidad de datos a largo plazo: Bitcoin (BTC), Ethereum (ETH), Litecoin (LTC), Dash (DASH) y Monero (XMR). Tras el filtro, el subconjunto de trabajo contiene 8,698 registros diarios, con coberturas distintas por moneda: Bitcoin y Litecoin desde abril de 2013, Dash desde febrero de 2014, Monero desde mayo de 2014 y Ethereum desde agosto de 2015, todas hasta noviembre de 2018. Esta diferencia de antigüedad entre monedas es relevante porque limita posteriormente el período en el que pueden compararse simultáneamente los cinco activos.

Pipeline de limpieza de datos

La limpieza se implementa en `limpiar_datos()` y sigue una secuencia ordenada de pasos:

1. Filtrado de monedas de interés: se conserva únicamente el subconjunto de las cinco criptomonedas seleccionadas, descartando el resto del dataset.
2. Conversión de tipos: la columna `date` se transforma a formato `datetime`, y las columnas numéricas (`open`, `high`, `low`, `close`, `volume`, `market`, entre otras) se fuerzan a tipo `float` con `pd.to_numeric(errors="coerce")`, de modo que cualquier valor no convertible se transforma en `NaN` en lugar de interrumpir la carga.
3. Eliminación de duplicados: se descartan filas exactamente repetidas con `drop_duplicates()`.
4. Eliminación de nulos en columnas clave: se eliminan registros con valores faltantes en `date`, `close`, `volume` o `market`, ya que son indispensables para el cálculo de métricas y retornos.
5. Descarte de valores no positivos: se eliminan filas donde el precio de cierre o el volumen sean iguales a cero o negativos, al representar errores de registro o días sin actividad real de mercado. En la práctica, este paso retiró 486 registros con volumen no positivo.
6. Ordenamiento: el DataFrame resultante se ordena por moneda y fecha, garantizando series cronológicas consistentes para los cálculos posteriores.

Como resultado, el dataset limpio pasa de 8,698 a 8,212 registros válidos, distribuidos así: Bitcoin (1,799), Litecoin (1,799), Dash (1,750), Monero (1,653) y Ethereum (1,211).

Construcción de la matriz de retornos

La matriz de retornos se construye en `calcular_retornos()` en dos etapas:

Pivoteo: el DataFrame limpio se transforma en una tabla pivote con `pivot_table()`, donde el índice es la fecha, las columnas son las monedas y los valores son el precio de cierre (`close`).

Cálculo de retornos logarítmicos: se calcula el retorno diario como $\ln(P_t / P_{t-1})$ para cada moneda, en lugar del retorno aritmético simple. Se prefieren los retornos logarítmicos porque son aditivos en el

tiempo y tienen mejor comportamiento numérico durante la optimización con `scipy.optimize.minimize`.

Un punto metodológico importante: como el pivoteo coloca a las cinco monedas en columnas y el `.dropna()` final elimina cualquier fecha en la que al menos una moneda no tenga dato, la matriz de retornos queda automáticamente acotada al período en que las cinco monedas coexisten. Dado que Ethereum es el activo más reciente (ingresa en agosto de 2015), la matriz de retornos resultante cubre únicamente 1,210 observaciones diarias, desde el 8 de agosto de 2015 hasta el 29 de noviembre de 2018, en lugar de los más de 1,700 registros disponibles para Bitcoin o Litecoin de forma individual. Esta matriz de $1,210 \times 5$ es la que alimenta directamente el cálculo de la matriz de covarianza y la posterior optimización de portafolio (Markowitz).

Datos en tiempo real (CoinGecko API)

Además de la base histórica, el dashboard incorpora una pestaña de datos en vivo que consulta la API pública de CoinGecko (sin necesidad de API key), implementada mediante `urllib.request` en tres funciones principales:

`_coingecko_get()`: función auxiliar que construye la URL de consulta (<https://api.coingecko.com/api/v3/...>) con los parámetros codificados vía `urlencode()`, envía la solicitud con cabeceras `accept: application/json` y `user-agent` personalizado, y parsea la respuesta JSON.

`obtener_precios_coingecko()`: consulta el endpoint `simple/price` para obtener, por cada moneda, el precio actual en USD, la variación porcentual de 24 horas, el volumen de 24 horas, la capitalización de mercado y la marca temporal de última actualización (`last_updated_at`, convertida a `datetime` con `pd.to_datetime(unit="s")`).

`obtener_market_chart_coingecko()`: consulta el endpoint `coins/{id}/market_chart` para obtener series recientes de 30 o 90 días (precio, volumen y market cap), introduciendo una pausa de 1.1 segundos entre solicitudes consecutivas (`time.sleep(1.1)`) para evitar el límite de tasa de la API pública (error HTTP 429). Los datos devueltos se normalizan al mismo esquema de columnas que el dataset histórico (`name`, `date`, `open`, `high`, `low`, `close`, `volume`, `market`), lo que permite reutilizar las mismas funciones de limpieza, cálculo de retornos y optimización de portafolio sobre los datos live.

Esta capa en tiempo real permite comparar el comportamiento histórico (2013–2018) con las condiciones actuales del mercado, calculando incluso un portafolio óptimo alternativo basado únicamente en los datos recientes de CoinGecko.

Implementación técnica

Arquitectura del proyecto

El proyecto sigue una arquitectura modular que separa con claridad las responsabilidades entre datos, lógica de negocio, presentación visual y verificación automática. La estructura del repositorio es la siguiente:

```
ProyectoProgramación/  
├── data/  
│   └── crypto-markets.csv    # Dataset histórico de criptomonedas  
├── src/  
│   ├── app.py               # Orquestador Streamlit (UI, estado, caché)  
│   ├── processing.py         # Lógica de negocio y datos  
│   └── viz.py                # Visualizaciones Plotly puras  
├── tests/  
│   └── test_processing.py    # Pruebas unitarias con pytest  
├── docs/  
│   └── parcial.pdf  
├── requirements.txt  
└── README.md
```

Esta separación responde al principio de responsabilidad única: cada módulo tiene un propósito bien delimitado y depende del anterior, pero no al revés.

processing.py concentra toda la lógica de negocio y acceso a datos: carga del CSV, limpieza, cálculo de métricas, retornos logarítmicos, optimización de portafolio (Markowitz) y las consultas a la API de CoinGecko. No conoce nada de Streamlit ni de la interfaz; solo recibe DataFrames o parámetros y devuelve estructuras de datos (DataFrames o diccionarios).

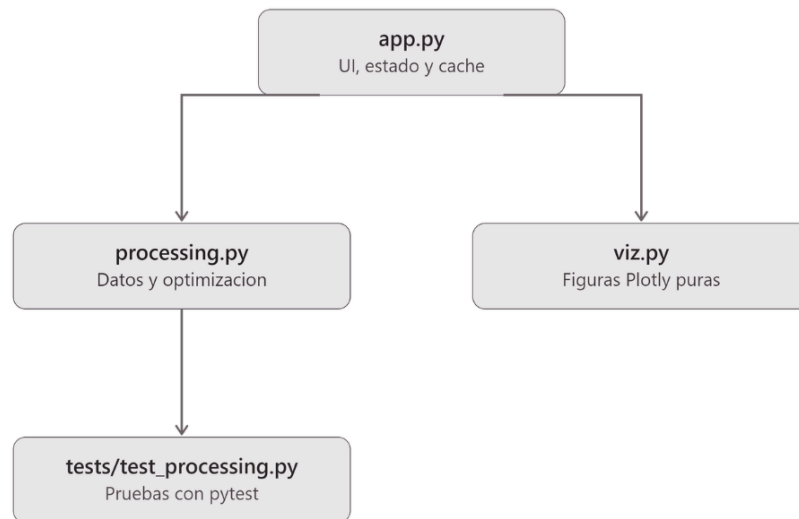
viz.py contiene exclusivamente funciones de graficación con Plotly. Cada función recibe datos ya procesados y devuelve un objeto `go.Figure`, sin modificar el estado de la aplicación ni realizar cálculos de negocio.

app.py actúa como orquestador: importa funciones de `processing.py` y `viz.py`, gestiona el estado de la sesión (`st.session_state`), define el cacheo (`st.cache_data`) y construye la interfaz (sidebar, pestañas, tarjetas KPI).

`tests/test_processing.py` valida el comportamiento de las funciones críticas de `processing.py` de forma aislada, sin depender de Streamlit ni de la interfaz.

Este diseño permite que la lógica de procesamiento se pueda testear de forma independiente, que las visualizaciones sean reutilizables y que la interfaz se mantenga simple, ya que delega todo el trabajo pesado a los otros dos módulos.

Diagrama de dependencias entre módulos:



Optimización del portafolio

La optimización del portafolio se implementa en la función `optimizar_portafolio()` de `processing.py`, siguiendo el Modelo de Markowitz.

Función objetivo según perfil. A diferencia de un modelo de mínima varianza fijo, esta implementación permite tres funciones objetivo distintas según el perfil de riesgo elegido por el usuario:

Conservador: minimiza la volatilidad anualizada del portafolio, $\sqrt{(w^T \Sigma w)}$.

Balanceado: maximiza el Sharpe ratio (se minimiza su negativo, ya que `scipy.optimize.minimize` solo minimiza).

Agresivo: maximiza el retorno esperado anual del portafolio (se minimiza su negativo).

Restricciones. En los tres perfiles se aplican las mismas restricciones de Markowitz: la suma de los pesos debe ser igual a 1 (`{"type": "eq", "fun": lambda w: np.sum(w) - 1}`) y cada peso individual está acotado al intervalo $[0, 1]$, lo que impide posiciones cortas (long-only).

Método y punto de inicio. La optimización usa el método `SLSQP` (Sequential Least Squares Programming) de `scipy.optimize.minimize`, adecuado para problemas con restricciones de

igualdad y límites de caja. El punto de inicio es un vector de pesos iguales, $w_0 = \frac{1}{n}$ para cada una de las n monedas seleccionadas, lo que garantiza un arranque neutral sin sesgar la búsqueda hacia ningún activo.

Parámetros de convergencia. Se fija una tolerancia $ftol = 1e - 12$ y un máximo de 1,000 iteraciones, lo que asegura una convergencia muy precisa sin comprometer el tiempo de cómputo dado el tamaño reducido del problema (5 variables).

Frontera eficiente. Para construir la frontera, se calcula primero el rango de retornos posibles (`ret_min` y `ret_max`, derivados de los retornos anualizados de cada moneda) y se generan 60 puntos equiespaciados entre ambos extremos con `np.linspace`. Para cada nivel de retorno objetivo se resuelve un subproblema de minimización de varianza, añadiendo una restricción de igualdad adicional que fija el retorno del portafolio en ese nivel. Los puntos (volatilidad, retorno) que logran convergencia exitosa (`res.success`) se almacenan y conforman la curva de la frontera eficiente.

Formulación matemática. El problema de optimización para el perfil conservador (mínima varianza) se expresa formalmente como:

$$\min_w \sqrt{w^T \Sigma w} \quad \text{sujeto a} \quad \sum_i w_i = 1, \quad w_i \in [0, 1]$$

donde w es el vector de pesos del portafolio, Σ es la matriz de covarianza anualizada de los retornos logarítmicos (multiplicada por 365), y la restricción $w_i \in [0, 1]$ impone la condición long-only.

Módulo de visualización

`viz.py` concentra 10 funciones, cada una responsable de generar un único tipo de gráfico Plotly a partir de datos ya procesados:

`grafico_precio_tiempo`: evolución del precio de cierre en el tiempo por moneda, con opción de escala logarítmica.

`grafico_velas`: gráfico de velas japonesas (OHLC) para una moneda individual.

`grafico_volumen`: volumen diario de transacciones de una moneda.

`grafico_volatilidad`: comparación de la volatilidad anual entre las monedas seleccionadas, a partir de la tabla de métricas.

`grafico_histograma_retornos`: distribución de los retornos diarios de una moneda.

`grafico_pesos_portafolio`: gráfico de pastel (pie chart) con la distribución de pesos del portafolio óptimo.

`grafico_frontera_eficiente`: curva de la frontera eficiente con el punto óptimo resaltado según el perfil de riesgo.

`grafico_live_precios`: series recientes de precios obtenidas desde CoinGecko (30 o 90 días).

`grafico_correlacion`: heatmap de correlación entre los retornos de las monedas seleccionadas.

`grafico_backtest`: curva de valor acumulado del backtesting, comparando el portafolio óptimo contra cada moneda individual.

Un rasgo de diseño clave de este módulo es que todas las funciones son puras: reciben datos (DataFrames, diccionarios o listas de puntos) y retornan un objeto `go.Figure` de Plotly, sin modificar variables externas, sin acceder a `st.session_state` y sin efectos secundarios (side effects). Esta pureza permite reutilizar cualquier función de graficación fuera de Streamlit, por ejemplo, en un notebook o en un script de exportación, y facilita enormemente las pruebas, ya que el resultado depende únicamente de los argumentos recibidos.

Gestión de estado y caché

El manejo del estado y del rendimiento se concentra en `app.py`, apoyándose en dos mecanismos nativos de Streamlit:

@st.cache_data. Se decoran cuatro funciones con `@st.cache_data`: `obtener_datos()` (carga y limpieza del CSV), `obtener_portafolio()` (que ejecuta la optimización completa, incluyendo el barrido de 60 iteraciones para la frontera eficiente), `obtener_live_snapshot()` y `obtener_live_chart()` (ambas consultan la API pública de CoinGecko). Sin este cacheo, cada interacción del usuario con la interfaz, cambiar un filtro, mover un slider, forzaría una relectura del CSV completo y una nueva optimización con SLSQP, lo cual sería costoso en tiempo y, en el caso de CoinGecko, podría disparar el límite de solicitudes (HTTP 429) de la API.

Botón "Actualizar precios". Este control, ubicado en el panel lateral, invoca `st.cache_data.clear()` seguido de `st.rerun()`. Esto invalida todas las cachés (no solo la de CoinGecko) y fuerza un nuevo cálculo, lo que es necesario porque CoinGecko no se actualiza automáticamente en cada render del dashboard, solo cuando el usuario lo solicita explícitamente o cuando cambian los parámetros de entrada de las funciones cacheadas.

st.session_state. Se utiliza para persistir la selección de criptomonedas (`monedas_sel`) entre interacciones del usuario. Al iniciar la sesión, si la clave `monedas_sel` no existe, se inicializa con la lista de monedas por defecto (`TOP5_DEFAULT`). El widget `st.multiselect` está vinculado a esta clave de estado, de modo que la selección del usuario persiste mientras navega entre pestañas, y existe además un botón de restauración (`restaurar_monedas_default()`) que vuelve a las cinco monedas originales si el usuario deselecta todas.

Testing

El archivo `tests/test_processing.py` contiene una prueba unitaria implementada con `pytest` que valida el comportamiento de dos funciones críticas de `processing.py`, usando un `DataFrame` de ejemplo construido manualmente (dos monedas, tres fechas cada una):

Ausencia de NaN en los retornos. Se llama a `calcular_retornos(df)` y se verifica con `retornos.isna().any().any()` que ninguna celda de la tabla pivote de retornos logarítmicos contenga valores nulos. Esto confirma que la limpieza previa (tipos, fechas, eliminación de la primera observación por el `dropna()` tras el `shift(1)`) deja la matriz lista para ser usada en la optimización sin provocar errores numéricos en `scipy.optimize`.

Los pesos óptimos suman 1.0. Se ejecuta `optimizar_portafolio(retornos, tasa_libre_riesgo=4.5)` y se comprueba que `abs(sum(portafolio["pesos"].values()) - 1.0) < 0.001`. Esta prueba válida que la restricción de igualdad ($\sum w_i = 1$) efectivamente se cumple en el resultado devuelto por el optimizador, dentro de una tolerancia razonable frente al error numérico propio del método SLSQP.

Cómo ejecutarlo. Desde la raíz del proyecto, con el entorno virtual activado y las dependencias instaladas (`pip install -r requirements.txt`), basta con ejecutar `pytest`.

`Pytest` descubre automáticamente el archivo `test_processing.py` por su prefijo `test_` y ejecuta la función `test_retornos_no_tienen_nan_y_pesos_suman_uno()`. Si ambas aserciones se cumplen, la salida confirma que la prueba pasó (1 passed); si alguna falla, `pytest` reporta el `assertion error` específico junto con la traza para facilitar la depuración.

Resultados y análisis

Análisis exploratorio de precios

El análisis cubre el período histórico 27 de diciembre de 2013 al 29 de noviembre de 2018, aunque la cobertura efectiva varía por moneda según su fecha de aparición en el mercado: Bitcoin y Litecoin tienen registros desde 2013-12-27, Dash desde 2014-02-14, Monero desde 2014-05-21 y Ethereum desde 2015-08-07, lo que se refleja en el número de observaciones por moneda (1,799 para Bitcoin y Litecoin, 1,750 para Dash, 1,653 para Monero y solo 1,211 para Ethereum).

KPIs históricos por moneda:

Moneda	Precio medio (USD)	Retorno total (%)	Volatilidad anual (%)
Bitcoin	2,562.82	482.1	75.22
Ethereum	213.41	4,143.32	141.39
Litecoin	34.47	45.25	116.4
Dash	123.72	25,743.26	194.32
Monero	54.06	3,725.0	149.54

El panel de indicadores clave del dashboard confirma este patrón: Bitcoin es, con diferencia, la moneda más estable del grupo (menor volatilidad anual, 75.22%), mientras que Dash combina el mayor retorno total del período (+25,743%) con la mayor volatilidad (194.32%). Este contraste anticipa el resultado central de la sección 5.3: un optimizador de mínima varianza tenderá a favorecer fuertemente a Bitcoin, no porque tenga el mejor retorno, sino porque su perfil de riesgo es el más bajo.

Evolución 2013 - 2018. El gráfico de evolución de precios (pestaña Precios & Mercado) muestra con claridad las dos fases que dominan la serie histórica completa:

Rally de 2017 - 2018: Bitcoin experimenta un crecimiento prácticamente vertical desde mediados de 2017, alcanzando su pico cercano a los \$20,000 USD en enero de 2018. El gráfico de volumen diario confirma que este movimiento estuvo acompañado por un salto igualmente abrupto en el volumen transaccionado, lo que es consistente con un episodio de especulación intensa más que con una apreciación gradual sostenida por fundamentos.

Crash de 2018: tras el pico de enero, el precio se desploma de forma sostenida durante el resto del año, cerrando la serie muy por debajo del máximo alcanzado. El gráfico de velas japonesas para Bitcoin permite observar este patrón día a día: velas verdes y alargadas durante

el ascenso, seguidas por una sucesión de velas rojas con cuerpos cada vez más amplios durante la corrección.

Volatilidad y retornos diarios. El gráfico de volatilidad anual comparativa ordena a las cinco monedas de mayor a menor riesgo: Dash (194%) > Monero (150%) > Ethereum (141%) > Litecoin (116%) > Bitcoin (75%). El histograma de retornos diarios de Bitcoin exhibe la forma típica de una distribución financiera leptocúrtica: la gran mayoría de las observaciones se concentra muy cerca de 0%, pero con colas visibles que reflejan los movimientos extremos asociados al rally y al crash. Esta forma de distribución, colas más pesadas que las de una normal, es precisamente la razón por la que medidas de riesgo basadas en la desviación estándar, como las usadas en el modelo de Markowitz, tienden a subestimar el riesgo de eventos extremos.

Análisis de correlaciones

El heatmap de correlación de retornos diarios entre las cinco monedas arroja la siguiente matriz:

	Bitcoin	Dash	Ethereum	Litecoin	Monero
Bitcoin	1.00	0.45	0.36	0.60	0.48
Dash	0.45	1.00	0.35	0.43	0.46
Ethereum	0.36	0.35	1.00	0.35	0.34
Litecoin	0.60	0.43	0.35	1.00	0.42
Monero	0.48	0.46	0.34	0.42	1.00

Mayor correlación. El par Bitcoin - Litecoin (0.60) presenta la correlación más alta de toda la matriz. Esto es consistente con que ambas son las criptomonedas más "establecidas" del grupo y con mayor antigüedad en el dataset, por lo que tienden a moverse de forma más sincronizada ante shocks de mercado generalizados (sentimiento de riesgo, regulación, flujos de capital hacia el sector cripto en su conjunto).

Menor correlación / mejor diversificador. Ethereum es la moneda con las correlaciones más bajas frente a todas las demás (0.36 con Bitcoin, 0.35 con Dash, 0.35 con Litecoin, 0.34 con Monero), con un promedio de correlación de 0.35, claramente por debajo del resto de monedas (Bitcoin 0.47, Litecoin 0.45, Monero 0.43, Dash 0.42 en promedio). Esto convierte a Ethereum en el activo más útil para diversificar dentro de este conjunto de cinco monedas.

Relación con Markowitz. Este resultado conecta directamente con el fundamento teórico de la sección 4.2: cuando dos activos tienen baja correlación, combinarlos en un portafolio reduce la varianza total por debajo del promedio ponderado de las varianzas individuales (efecto de diversificación). Sin embargo, es importante notar una tensión que aparece en los resultados de la sección 5.3: aunque Ethereum es el mejor diversificador por correlación, el optimizador de mínima varianza no le asigna un peso proporcional a ese beneficio, porque su volatilidad individual (141.39%) sigue siendo muy alta. El modelo de mínima varianza pondera ambos efectos simultáneamente, covarianza y varianza individual, y en este caso el efecto de la volatilidad individual de Bitcoin domina sobre el efecto de diversificación de Ethereum.

Portafolios óptimos por perfil de riesgo

Con los parámetros configurados en el dashboard (tasa libre de riesgo anual de 4.50%, las cinco monedas seleccionadas), el perfil conservador produce el siguiente resultado:

Métrica	Valor
Volatilidad anual	73.70%
Retorno esperado anual	85.97%
Sharpe ratio ($r_f = 4.50\%$)	1.105

Distribución de pesos - perfil conservador:

Moneda	Peso (%)	Volatilidad anual (%)	Retorno total histórico (%)
Bitcoin	75.7	75.22	482.1
Dash	14.56	194.32	25,743.26
Ethereum	5.31	141.39	4,143.32
Litecoin	4.42	116.4	45.25
Monero	0.0	149.54	3,725.0

Por qué el modelo asigna estos pesos. El optimizador concentra el **75.7%** del capital en Bitcoin porque, al minimizar la volatilidad del portafolio, el criterio dominante es el riesgo individual de cada activo: Bitcoin tiene, por amplio margen, la menor volatilidad anual (75.22% frente a 116–194% del resto). Los pesos residuales en Dash (14.56%), Ethereum (5.31%) y Litecoin (4.42%) no se explican

por su propia volatilidad, todas son más volátiles que Bitcoin, sino por el efecto de diversificación derivado de sus correlaciones parciales con Bitcoin: incluso una pequeña proporción de un activo poco correlacionado puede reducir la varianza total del portafolio. Monero recibe un peso de 0%: combina una volatilidad alta (149.54%) con una correlación relativamente alta frente a Bitcoin (0.48, la segunda más alta después de Litecoin), por lo que no aporta beneficio marginal de diversificación suficiente para justificar incluirlo, dado el conjunto de activos disponible.

El punto resultante se ubica, como se observa en el gráfico de la frontera eficiente, en el extremo izquierdo de la curva: el punto de menor volatilidad alcanzable (~73.7%) para un retorno esperado de ~86%.

Backtesting del portafolio óptimo

El backtesting simula el valor acumulado de una inversión inicial de \$1,000 USD distribuida según los pesos óptimos del perfil conservador, comparada contra invertir el mismo monto íntegramente en cada moneda individual (estrategia buy-and-hold), durante todo el período histórico (2013–2018).

Se muestra que durante el rally de 2017–2018, los activos individuales de mayor volatilidad (particularmente Dash y Ethereum) alcanzan picos de valor acumulado muy superiores a los del portafolio diversificado, lo que es coherente con su retorno total histórico extremo (+25,743% y +4,143% respectivamente). Sin embargo, esa misma volatilidad implica caídas mucho más pronunciadas durante la corrección de 2018: las curvas de los activos individuales muestran retrocesos abruptos desde sus máximos, mientras que la curva del portafolio óptimo se mantiene visiblemente más suave a lo largo de todo el período, sin alcanzar los picos extremos pero también sin sufrir las caídas más severas.

Este es el resultado esperado de la teoría de Markowitz: el portafolio de mínima varianza no maximiza el retorno en los períodos alcistas, sino que protege mejor del drawdown en los períodos de corrección, a costa de renunciar a parte de la ganancia potencial de los activos más volátiles.

Precios en tiempo real

La pestaña Live CoinGecko muestra el snapshot de mercado consultado al 24 de junio de 2026, 14:11 UTC:

Moneda	Precio actual (USD)	Variación 24h
Bitcoin	61,283	-1.85%
Ethereum	1,646	-0.93%

Litecoin	41.55	-3.09%
Dash	35.08	+0.50%
Monero	321.51	+0.50%

Comparación con el período histórico. La diferencia de escala frente al período 2013–2018 es notable: Bitcoin pasó de un precio medio histórico de ~\$2,563 a cotizar actualmente por encima de los \$61,000, lo que evidencia cuánto ha madurado el mercado desde el cierre del dataset histórico (noviembre 2018) hasta hoy. El gráfico de precios recientes (ventana de 30 días) muestra además una tendencia bajista entre el 23 de mayo y el 8 de junio de 2026, con Bitcoin retrocediendo desde cerca de \$75,000 hasta ubicarse alrededor de \$61,000.

Portafolio óptimo con datos live. Al recalcular el portafolio de mínima varianza usando únicamente la ventana reciente de 30 días, el resultado cambia drásticamente frente al histórico: volatilidad de 43.05%, un retorno anualizado de -266.83% y un Sharpe de -6.303. Estos valores, aunque parezcan alarmantes, son un artefacto esperable de anualizar una tendencia bajista de muy corto plazo (30 días): cualquier caída sostenida en una ventana tan corta, al multiplicarse por el factor de anualización ($\sqrt{365}$), produce retornos y Sharpe ratios extremos que no deben interpretarse de la misma forma que las métricas calculadas sobre 5 años de historia. Esto resalta una limitación importante que conviene discutir en las conclusiones: el modelo de Markowitz es sensible al horizonte temporal usado para estimar μ y Σ , y una ventana corta puede producir estimaciones de retorno esperado muy ruidosas.

Valor técnico de la integración en tiempo real. Más allá de los números puntuales, esta pestaña demuestra que la arquitectura del proyecto permite comparar mercado pasado y mercado actual sin reescribir lógica: las mismas funciones `calcular_retornos()` y `optimizar_portafolio()` de `processing.py` se reutilizan sin modificación, alimentadas esta vez con datos de `obtener_market_chart_coingecko()` en lugar del CSV histórico.

Conclusión

El proyecto logró desarrollar un dashboard interactivo funcional para el análisis de portafolios de criptomonedas, cumpliendo los objetivos planteados al inicio del documento. Se construyó un pipeline de datos completo y validado (carga, limpieza y cálculo de métricas) que procesa de forma robusta el dataset histórico de cinco criptomonedas entre 2013 y 2018, descartando registros inválidos y garantizando consistencia en los tipos de datos.

Sobre la base de la Teoría Moderna de Portafolios de Markowitz, se implementó un módulo de optimización capaz de calcular no solo el portafolio de mínima varianza, sino también variantes para tres perfiles de riesgo, Conservador (mínima volatilidad), Balanceado (máximo Sharpe ratio) y Agresivo (máximo retorno esperado), además de la construcción de la frontera eficiente mediante 60 puntos de barrido. Esto amplía el alcance original del análisis, permitiendo comparar cómo distintos criterios de optimización conducen a distribuciones de pesos muy diferentes dentro del mismo conjunto de activos.

El portafolio de mínima varianza obtenido (perfil Conservador) sí logra reducir el riesgo respecto a mantener cualquier criptomoneda de forma individual: alcanza una volatilidad anual de 73.70%, por debajo incluso de Bitcoin (75.22%), que es el activo individualmente menos volátil del conjunto. Este resultado confirma empíricamente el principio central de Markowitz: combinar activos con correlaciones parciales bajas entre sí, como se observa en la matriz de correlación calculada, con valores entre 0.34 y 0.6, reduce el riesgo total de la cartera por debajo del promedio ponderado de los riesgos individuales, incluso cuando uno de los activos (Bitcoin) domina el portafolio con un 75.7% de los pesos.

A nivel de arquitectura, el proyecto separó claramente la lógica de procesamiento de datos (`processing.py`), la capa de visualización (`viz.py`) y la interfaz de usuario (`app.py`), lo que facilitó la incorporación de pruebas automatizadas con `pytest` que verifican la ausencia de valores nulos en los retornos calculados y la correcta normalización de los pesos del portafolio (suma igual a 1). Adicionalmente, se integró una fuente de datos en tiempo real a través de la API pública de CoinGecko, lo que permitió extender el análisis histórico con una pestaña live que recalcula el portafolio óptimo sobre ventanas recientes de 30 o 90 días, ofreciendo una comparación directa entre el comportamiento pasado y las condiciones actuales del mercado.

El dashboard demuestra que la diversificación, incluso entre activos de alta volatilidad individual como las criptomonedas, sigue siendo una herramienta efectiva para la gestión de riesgo cuando se aplica un marco cuantitativo riguroso como el de Markowitz.

Trabajo futuro

A partir de los resultados obtenidos, se identifican las siguientes líneas de mejora para futuras iteraciones del proyecto:

- **Migración completa a datos de CoinGecko.** Actualmente el análisis histórico depende del dataset estático de Kaggle (2013–2018). Una mejora natural es descargar el historial completo

de cada moneda directamente desde la API de CoinGecko, eliminando la dependencia de un archivo CSV desactualizado y permitiendo que el dashboard se mantenga vigente sin necesidad de actualizar manualmente los datos.

- **Tasa libre de riesgo configurable con mayor granularidad.** Si bien el dashboard ya permite ajustar la tasa libre de riesgo anual desde el panel lateral, se podría extender esta funcionalidad para obtenerla automáticamente desde una fuente externa (por ejemplo, el rendimiento de bonos del Tesoro de EE. UU.), evitando que el usuario deba ingresarla manualmente y mejorando la precisión del Sharpe ratio calculado.
- **Incorporación de más criptomonedas.** El análisis actual se limita a cinco activos seleccionados por su relevancia histórica. Ampliar el universo de inversión, incluyendo stablecoins, tokens de capitalización media o activos DeFi, permitiría evaluar el efecto de la diversificación en un espacio de optimización más amplio y realista.
- **Análisis de drawdown máximo.** El dashboard actual reporta volatilidad, retorno y Sharpe ratio, pero no captura el riesgo de pérdidas sostenidas en el tiempo. Incorporar el cálculo del drawdown máximo (la mayor caída desde un pico hasta un valle) complementaría las métricas de riesgo existentes, especialmente relevante dada la alta volatilidad observada en activos como Dash y Monero.
- **Exportación ampliada de resultados.** El dashboard ya permite descargar la tabla de pesos óptimos como CSV; una extensión natural sería permitir también la descarga de la frontera eficiente completa, el histórico de backtesting y un reporte consolidado en PDF, facilitando que el análisis pueda compartirse o auditarse fuera de la aplicación.

Referencias bibliográficas

Markowitz, H. (1952). *Portfolio Selection*. *The Journal of Finance*, 7(1), 77–91.
<https://doi.org/10.2307/2975974>

Event, J. (2018). *All Crypto Currencies — CoinMarketCap Historical Data [Dataset]*. Kaggle. <https://www.kaggle.com/datasets/jessevent/all-crypto-currencies>

Streamlit Inc. (2024). *Streamlit Documentation — Build and share data apps*.
<https://docs.streamlit.io>

The Pandas Development Team. (2024). *pandas: powerful Python data analysis toolkit (v2.2.2)*. <https://pandas.pydata.org/docs>

Harris, C. R., Millman, K. J., van der Walt, S. J., et al. (2020). *Array programming with NumPy*. Nature, 585, 357–362. <https://doi.org/10.1038/s41586-020-2649-2>

Virtanen, P., Gommers, R., Oliphant, T. E., et al. (2020). *SciPy 1.0: Fundamental algorithms for scientific computing in Python*. Nature Methods, 17, 261–272. <https://doi.org/10.1038/s41592-019-0686-2>

Plotly Technologies Inc. (2024). *Plotly Python Open Source Graphing Library* (v5.22). <https://plotly.com/python>

Anexo

Para el desarrollo de este proyecto se utilizó Claude (claude.ai), asistente de inteligencia artificial desarrollado por Anthropic, como herramienta de apoyo en los siguientes aspectos:

Scaffolding inicial de los módulos principales: Se solicitó apoyo para generar la estructura base de los archivos `processing.py`, `viz.py` y `app.py`, incluyendo la organización de funciones, el diseño visual del dashboard, la paleta de colores oscura, las tarjetas KPI, el panel lateral de navegación y los gráficos interactivos con Plotly.

Redacción del informe: Se utilizó Claude como asistente de redacción para estructurar y elaborar las secciones del presente documento, tomando como insumo el código fuente del proyecto, el README, el dataset y las capturas del dashboard funcionando.

Sugerencias de estructura del repositorio: Se solicitó orientación sobre la organización de carpetas y archivos del proyecto, así como las buenas prácticas para separar la lógica de procesamiento, visualización y la interfaz principal.

Prompt 1 - Scaffolding del dashboard:

"Tengo un dataset de precios históricos de criptomonedas (Bitcoin, Ethereum, Litecoin, Dash y Monero) en un archivo `crypto-markets.csv`. Necesito construir un dashboard en Streamlit con diseño oscuro que incluya: un panel lateral con filtros por moneda y rango de fechas, KPIs por moneda, gráfico de evolución de precios, velas japonesas, volumen diario, volatilidad comparativa, histograma de retornos y una pestaña de portafolio óptimo con la frontera eficiente de Markowitz."

Prompt 2 - Redacción del informe:

"Ayúdame a redactar el informe de mi proyecto de herramientas de programación. La estructura es: descripción del proyecto, dataset, procesamiento de datos, dashboard Streamlit, portafolio de mínima varianza, estructura del repositorio y referencias. Los archivos que te estoy pasando son los archivos de mi web, incluyendo la base de datos, `requirements.txt`, `app.py` y el README. Dame las secciones una por una."

Prompt 3 - Integración de datos en vivo (CoinGecko):

"Ya tengo el dashboard funcionando con datos históricos. Ayúdame a agregar una pestaña que consulte la API pública de CoinGecko para traer precio actual, variación de 24h,

volumen y market cap de las mismas cinco monedas, además de una serie de precios recientes de 30 o 90 días."